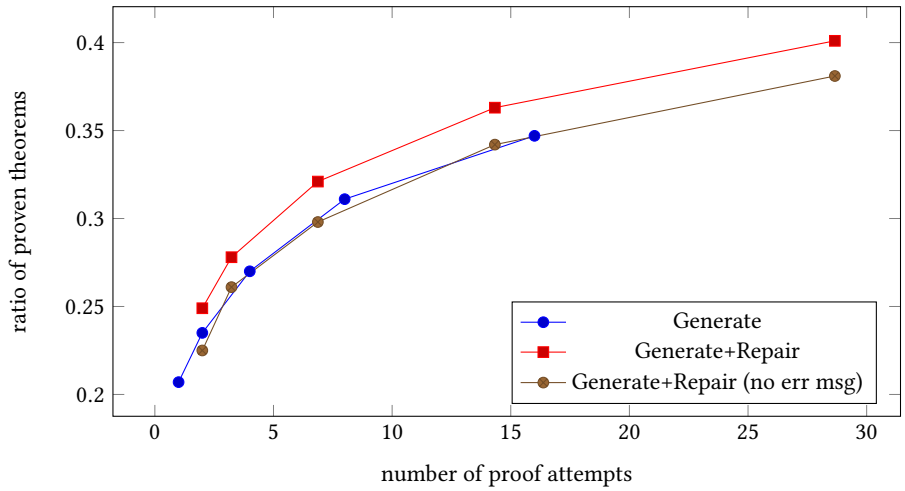

Figure 5: Ratio of theorems proven vs inference cost.

suggests that the information in the error message is crucial for the observed gains of the repair approach.

> **RA2**: LLMs can be used to repair proofs, including their own failed proof attempts, and this can boost overall proving power.

3.5 RQ3: Can LLMs benefit from using the context of the theorem?

In Table 4, we report the impact of adding theory file context to our plain generation approach. At 64 samples, the proof rate increases from 40.7% to 47.5% for the same model size. In Figure 6, we plot the proof success rate of the generation model with and without context against the number of proof attempts. We observe that the proof generation models with context consistently outperform the plain generation model.

To get a better understanding of where these gains are coming from, we inspected 5 randomly sampled examples that the model using context was able to solve, but the plain generation model could not. Appendix A displays these examples and further details the process we used to select them.

While the sample size is not large enough to make quantitative judgements, it appears that the model frequently makes use of similar proofs in the context. We observe that for 3 of the 5 examples (see Appendices A.1, A.3, A.5) the model readily **copies and adapts** proofs that exist in its context. For another example (see Appendix A.2), the model made use of a premise that did not occur in its context, which happened to also be used in the ground truth proof, but with a different tactic. In the final example (see Appendix A.4), the model found a simpler proof that did not occur like this in the context. This suggests that the addition of context does not play the same role as premise selection.

> **RA3**: LLMs can benefit from the context in which the theorem occurred in the theory file, both quantitatively by increasing proving power, and qualitatively by copying and adapting nearby proofs.

3.6 RQ4: Does the size of the LLM affect proof synthesis effectiveness?

We fine-tuned and evaluated the 62b version of Minerva on the proof generation task with context. In Table 4, we report that for 16 samples, the large model can prove an additional 1.3% over the 8b model, resulting in a total proof rate of 42.2%. For 64 samples, the large model can prove an additional 0.4% over the 8b model, resulting in a total proof rate of 47.9%.

In Figure 6, we plot the proof success rate of the generation model with context for the 8b model and the 62b model against the number of proof attempts. We observe that the 62b proof generation model with context outperforms the 8b proof generation model with context. One caveat here is that we were not able to tune hyperparameters as well due to the higher cost of these experiments, so an optimally tuned 62b model may perform even better.

> **RA4**: Theorem proving performance improves with the scale of the language model.

3.7 RQ5: How do LLMs compare to other SOTA proof generation methods?

While comparisons across different neural theorem provers are hard in general, we can compare to Thor [33], one of the strongest approaches available. Thor also relies on language models, but uses smaller models (700m parameters) and uses a different kind of proof step as its prediction target. Instead of using the human ground truth proofs, Thor generates a new training set and aims to solve each proof step by generating a declarative statement, which is

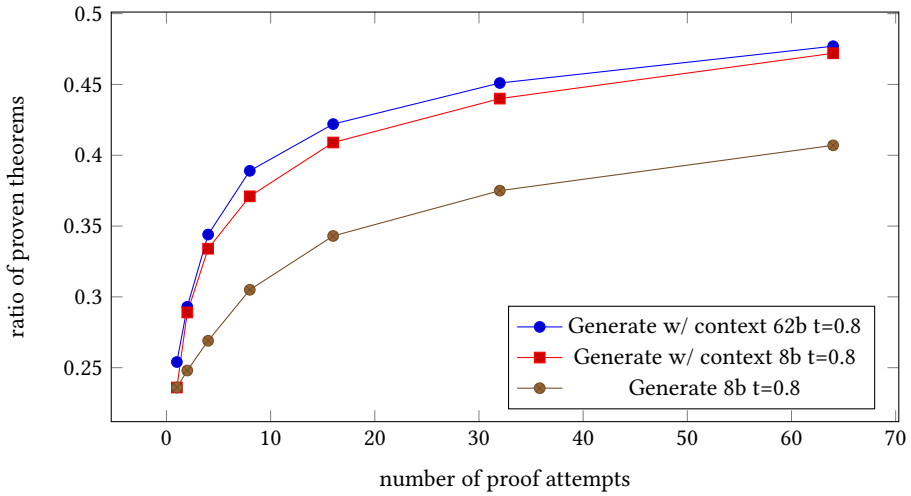

Figure 6: Ratio of theorems proven vs inference cost for models with different sizes and temperatures.

AFP Topic	Test set	Baldur	Thor
Computer Science	4,019	50.0%	57.5%
Logic	966	51.6%	53.6%
Mathematics	2,200	41.9%	50.5%
Tools	102	53.9%	51.8%

Table 7: Proof rate by AFP topic classification, and the number of theorems in each category. While there are only 6336 theorems in total in the test set, the projects these theorems appear in can fall into multiple topics.

then solved using Sledgehammer. That is, Thor disentangles the planning stage of the next proof step, which is the specification of the target state (using a "have" statement) and premise selection, which is done by Sledgehammer. This enables Thor to solve a total of 57% of the problems.

In contrast, our approach solves up to 47.9% of the problems. While there is a significant gap, we argue that the means by which the two techniques improve over plain language modeling are largely orthogonal. In Table 4, we report a large gain from 57% to 65.7% when we consider the union of Baldur and Thor, which supports this hypothesis.

We compare the proof rate of Baldur and Thor on different types of problems. The AFP is indexed by topic and there are four overarching topics: computer science, logic, mathematics, and tools. The authors of individual proof developments self-identity which topics their projects fall into. We use these provided topic labels to determine the categories of problems from our test that Baldur and Thor can most effectively solve. Table 7 shows the breakdown of which theorems in the test set fall into which topics and Baldur's and Thor's proof success rates on these theorems. In terms of relative performance, Baldur performs better than Thor on problems related to tools and similarly on problems related to logic. We observe that Baldur's performance on mathematics and computer science is less than that of Thor's performance. For mathematics proofs, we hypothesize that premise selection may be particularly useful, and Thor's use of Sledgehammer is likely what gives it a leg up on solving these mathematics problems.

> **RA5**: Our findings suggest that LLM-based methods and search-based methods are complementary, and together can lead to large gains in proving power.

4 DISCUSSION: WHAT'S NEXT?

Our evaluation shows that LLMs can generate whole proofs at once, and can repair their own mistakes, forming the basis for an effective and simple approach to proof synthesis. Moving forward, we find three directions particularly promising:

(1) integrating proof generation and proof repair models into a new **learnable proof search** strategy,
(2) investigating **alternative data splits** corresponding to different goals, and
(3) evaluating these techniques across **different proof assistants**.

Learnable Proof Search. While our generate + repair approach to proof synthesis lets us avoid costly proof search procedures, it also lends itself to a new proof search strategy. The search strategy would work as follows:

(1) use the generation model to sample candidate proofs,
(2) use the repair model to attempt to repair those proofs, and
(3) continue to use the repair model to repair the repair-model-generated attempts from (2).

This paves the way for a learnable proof search strategy.

We demonstrate a proof-of-concept of this new proof search strategy. We sample once using the generation model, repair the generated sample using the repair model, and repair the repair model's attempt using the repair model. When using both models, we sample with temperature 0. So the inference cost in this setup

is 3 (1 for the first generation, 1 for the first repair, and 1 for the second repair).

The generate + repair approach with inference cost of 2 proves 24.9% of the test set theorems. With a second repair attempt, it proves an additional 1.3%, for a total of 26.2%. The generation approach with inference cost of 3 proves 25.4%, which is 0.8% less than the second repair attempt for the same inference cost.

To make this a more viable proof search strategy, future work needs to focus on generating proof repair training data that better mirrors the required changes for the subsequent repair attempts. When proof checking, the resulting error message is for the first occurring error, typically from the first couple of lines of the predicted proof. So the proof repair model will only learn to address these types of errors. An alternative approach could be, for example, to take the training examples from the proof generation model and use the first few lines of the human-written ground truth proof as a *proof prefix*. We could then concatenate this proof prefix to the end of the input. Since it is a decoder-only model, we can simply sample the model's attempt at the rest of the proof. If the proof prefix concatenated with the rest of the proof does not check, then that can serve as a new training example for the proof repair model.

Alternative Data Splits. The PISA benchmark that we use to evaluate our approach commits to a particular data split between training data and testing data. It is interesting to note, however, that different data splits may themselves correspond to different goals, even fixing the same evaluation task and metric. Moving forward, it may be useful to consider different kinds of data splits corresponding to different goals, even fixing the same dataset and benchmark suite. Here, we consider two different splits: *theorem-wise* and *project-wise*.

PISA uses a random theorem-wise split of the theorems appearing the AFP. This means that for any theorem in the test set, the theorems and (the corresponding proofs) that appear before or after that theorem may be in the training set. This split is useful to evaluate since a forward-looking goal of neural theorem prover researchers is to integrate these tools directly into proof assistants, where they could make use of the full project context. That project context may include human-written proofs of nearby theorems that look similar (or even identical) to one another — automatically repurposing and adapting those proofs can be quite fruitful.

By contrast with PISA, CoqGym [96], the neural theorem prover benchmark suite for the Coq proof assistant, uses a project-wise split, where training and testing data come from entirely different projects. This is useful when the goal is to help proof engineers who start completely new projects and want an automated proof synthesis tool to prove as much as it can. A tool that is trained and evaluated in a setting where it expects that it has seen proofs in a given proof development, as may happen with a theorem-wise split, may not perform as well in this new setting. Explicit consideration for the data split and the goals it achieves may help drive neural theorem proving research even further.

Different Proof Assistants. To make better sense of new strides in neural theorem proving, it makes sense to evaluate the same techniques across many different proof assistants. But this remains challenging. Consider once again the problem of data splits: since prover developments that evaluate on CoqGym [21, 22] follow the same project-wise split as CoqGym, it can be hard to make sense of how those developments compare to those trained and evaluated using theorem-wise data splits, like our own Baldur.

We used an established benchmark of Isabelle/HOL proofs to fairly compare Baldur to prior work and to increase the chances that our results generalize. However, we observed that search-based proof-synthesis tools for other proof assistants tend to prove a smaller fraction of theorems than we have found in our work. For example, Diva [21], the current state of the art for the Coq proof assistant, proves 33.8% of its benchmark automatically. This could be a reflection of size and quality of the available training data or the complexity of the available evaluation data (which, by necessity, is different from what we use because it involves theorems and proofs in different languages), or a more fundamental difference in the complexity of synthesizing proofs in these respective languages.

Future work should allow for direct comparisons by porting the developed techniques across proof assistants. Cross-proof-assistant benchmark suites may help substantially with this, but still have their limitations. For example, MiniF2F [101] implements the same benchmark suite for Math Olympiad problems across many different proof assistants. But math problems are not evenly represented across proof assistants, which draw different user communities with different emphases. Fair comparisons between proof assistants are hard, but we do believe they are necessary.

5 RELATED WORK

Existing methods for automating formal theorem proving can be classified into two categories, hammers and search-based methods. Hammers, such as CoqHammer [17] and Sledgehammer [64], iteratively use a set of precomputed mathematical facts to attempt to "hammer" out a proof. While hammers are powerful, they lack the ability to employ certain tactics, such as induction, preventing them from proving certain large classes of theorems. Search-based methods use a prediction model that, given some information about a partially written proof, the target theorem being proven, and the current proof state, predicts a set of next likely proof steps. The methods then use metaheuristic search [28] to attempt to synthesize a proof. They iterate querying the prediction model for the likely next steps and using the proof assistant to get feedback on those steps and prune non-promising paths, generating a search tree of possible proofs. The proof assistant also determines when the proof is complete. The tools mostly differ in the prediction model they use, which are typically learned automatically. For example, ASTactic uses only the proof state [96], TacTok uses the proof state and the partially written proof script [22], Diva (which combines the use of many models) also uses the proof term [20], and Passport also uses identifier information [76]. Other search-based techniques include Tactician [5], Proverbot9001 [75], and GamePad [31] for Coq; TacticToe [23] for HOL4; and DeepHOL [4, 62] for HOL Light. Prior work has found that hammers and search-based methods are complementary, each often proving theorems the other cannot [20, 22, 96]. Thor [33] combines a search-based method with a hammer, using both a prediction model and Sledgehammer in its search. In contrast, our approach uses an LLM to generate an entire proof at once, and then to one-shot repair it.